

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 5

**з дисципліни «Створення примітивного кешуючого проксі-сервера на базі модуля
node:http»**

Виконав студент 2 курсу групи ФЕП-23

Накладюк Володимир Тарасович

Перевірив

асистент кафедри СП

Кулик Петро Русланович

Львів – 2026

Операційна система	Fedora 42
Репозиторій	http://github.com/3efirok/web_backend_lab5

Мета роботи

Метою лабораторної роботи є опанування вбудованого модуля **node:http**, практична перевірка знань про протокол HTTP, структуру HTTP-запиту та HTTP-відповіді, а також реалізація примітивного кешуючого проксі-сервера з використанням асинхронної роботи з файлами та зовнішнім ресурсом **https://http.cat**.

Короткий опис реалізації

У межах лабораторної роботи було створено простий HTTP-сервер у файлі `main.js`. Під час запуску програма перевіряє параметри командного рядка, зчитує адресу сервера, порт та шлях до директорії кешу. Якщо теки кешу не існує, вона створюється автоматично.

Сервер обробляє запити, у шляху яких передається HTTP-код, наприклад `/200` або `/404`. Для кожного такого коду в кеші використовується окремий файл формату `.jpg`.

При обробці методу `GET` сервер спочатку читає картинку з диска. Якщо файл знайдено, він повертається клієнту із заголовком `Content-Type: image/jpeg`. Якщо файлу немає, сервер звертається до `https://http.cat/<код>`, завантажує зображення, зберігає його в кеш та надсилає клієнту.

Метод `PUT` записує зображення з тіла запиту в кеш і повертає `201 Created`. Метод `DELETE` видаляє файл із кешу і повертає `200 OK`. Для інших HTTP-методів сервер повертає `405 Method Not Allowed`.

Основні фрагменти роботи програми

Параметри командного рядка

Для читання параметрів командного рядка у програмі використовується пакет `Commander.js`. Сервер приймає три обов'язкові параметри:

- **-h, --host** — адреса сервера;
- **-p, --port** — порт сервера;
- **-c, --cache** — шлях до директорії кешу.

Якщо хоча б один із параметрів не передано, програма виводить повідомлення:

Please specify required arguments

Після цього виконання програми завершується.

Запуск сервера виконується командою:

```
npm start -- -h 127.0.0.1 -p 8080 -c ./cache
```

Для запуску в режимі автоматичного перезапуску використовується:

```
npm run dev -- -h 127.0.0.1 -p 8080 -c ./cache
```

Робота з кешем

Для роботи з файлами кешу використано асинхронні функції модуля **fs**:
fs.promises.readFile, **fs.promises.writeFile** та **fs.promises.unlink**.

Кеш зберігається у директорії, шлях до якої передається через параметр **--cache**. Якщо така директорія не існує, програма автоматично створює її за допомогою **fs.mkdirSync** з параметром **{ recursive: true }**.

Для кожного HTTP-коду у кеші створюється окремий файл у форматі:

<код>.jpg

Наприклад, для запиту **/200** файл кешу матиме назву:

200.jpg

Отримання зображень з http.cat

Якщо при виконанні **GET**-запиту потрібного зображення немає у кеші, сервер надсилає запит до зовнішнього ресурсу:

https://http.cat/<код>

Для цього використовується пакет **superagent**. Отримане зображення зчитується як **binary**-дані, після чого зберігається у кеш за допомогою **fs.promises.writeFile** і повертається клієнту.

Під час повторного запиту за тим самим HTTP-кодом сервер вже не звертається до **http.cat**, а бере файл безпосередньо з локального кешу. У такому випадку в консоль виводиться повідомлення:

Cache hit: <код>

Тестування програми

Для перевірки роботи сервера було використано браузер та утиліту **curl**.

Основні команди тестування:

```
curl -sS http://localhost:8080/200 --output 200.jpg
```

```
curl -sS http://localhost:8080/404 --output 404.jpg
```

```
curl -sS -X PUT --data-binary @200.jpg  
http://localhost:8080/200 -o /dev/null -w "%{http_code}\n"
```

```
curl -sS http://localhost:8080/200 --output  
200-from-cache.jpg
```

```
curl -sS -X DELETE http://localhost:8080/200 -o /dev/null  
-w "%{http_code}\n"
```

```
curl -sS -X POST http://localhost:8080/200 -o /dev/null -w  
"%{http_code}\n"
```

```
curl -sS http://localhost:8080/9999 -o /dev/null -w  
"%{http_code}\n"Перевірка: GET /200
```

Очікуваний результат: Повертається JPEG-зображення, статус 200.

Перевірка: GET /404

Очікуваний результат: Повертається JPEG-зображення для коду 404, статус 200.

Перевірка: PUT /200

Очікуваний результат: Файл записується в кеш, сервер повертає статус 201.

Перевірка: DELETE /200

Очікуваний результат: Файл видаляється з кешу, сервер повертає статус 200.

Перевірка: POST /200

Очікуваний результат: Сервер повертає статус 405 Method Not Allowed.

Перевірка: GET /9999

Очікуваний результат: Сервер повертає статус 404 Not Found, якщо зображення не знайдено.

Місце для скріншота 3:

Вставити скріншот сторінки:

<http://localhost:8080/404>

У браузері має відображатися картинка з сайту http.cat для HTTP-коду 404.

9. Висновок

Під час виконання лабораторної роботи було створено примітивний кешуючий проксі-сервер на базі модуля **node:http**. У результаті було реалізовано обробку HTTP-методів **GET**, **PUT** та **DELETE**, організовано зберігання зображень у файловому кеші, а також налаштовано отримання картинок із сервісу **http.cat** у разі відсутності даних на диску. Додатково було підготовлено запуск через **Docker Compose**, що робить проєкт зручним для перенесення на інші машини. У ході роботи було закріплено навички створення HTTP-сервера, роботи з асинхронними викликами, файловою системою, командним рядком та контейнеризацією.